

TECHNICAL DESIGN DOCUMENT & SYSTEM SPECIFICATION

Project Title: TacitIQ

Enterprise-Inspired Industrial Document Intelligence & Knowledge Graph Platform

Problem Statement (PS-8)

AI for Industrial Knowledge Intelligence: Unified Asset & Operations Brain

Submitted By

Team Name: ZeroToOne

Team Members:

- Sasidaran S
- Trishanth Sai S Y
- Pradish V

Department of Information Technology

SRM Institute of Science and Technology

Ramapuram Campus

Academic Year: 2026

Table of Contents

1. **Cover Page**
2. **Executive Summary**
3. **Problem Statement**
 - 3.1 Industrial Challenges
 - 3.2 Knowledge Loss and Retention
 - 3.3 Equipment Failures and Telemetry Gaps
 - 3.4 Unstructured Document Silos
 - 3.5 Business and Safety Implications
4. **Objectives**
5. **Proposed Solution**
6. **System Architecture**
 - 6.1 Overall Architecture
 - 6.2 Backend Services & Spring Boot
 - 6.3 Frontend Application (React & Vite)
 - 6.4 Hybrid Database Architecture (PostgreSQL & Neo4j)
 - 6.5 AI Query Processing Pipeline
 - 6.6 Document Parsing and Metadata Extraction Pipeline
 - 6.7 Knowledge Graph Causal Network Architecture
 - 6.8 JWT Authentication and Session Management Flow
7. **Technology Stack**
 - 7.1 Core Development Stack
 - 7.2 Database & Analytics Engine
 - 7.3 Libraries & Frameworks
8. **Non-Functional Requirements**
9. **Major Features**
 - 9.1 JWT Authentication
 - 9.2 Operations Assistant
 - 9.3 Digital Twin 3D
 - 9.4 Asset Dashboard
 - 9.5 Predictive Maintenance
 - 9.6 Knowledge Graph
 - 9.7 Workforce Planning
 - 9.8 Industrial Document Intelligence
 - 9.9 Document Metadata Extraction
 - 9.10 RAG-based Document Retrieval
10. **System Workflow**
11. **Database Design**
 - 10.1 PostgreSQL Entity-Relationship Model

- 10.2 Neo4j Label and Relationship Properties

12. REST API Overview

13. Implementation Details

14. Challenges Faced

15. Testing & Validation

16. Results

17. Future Enhancements

18. Advantages of TacitIQ

19. Conclusion

1. Cover Page

Project Title: TacitIQ: Enterprise-Inspired Industrial Document Intelligence & Knowledge Graph Platform

Problem Statement: Unifying Unstructured SOP Silos with Live IoT Telemetry and Causal Knowledge Graphs to Eliminate Maintenance Blindspots and Mitigate Workforce Retirement Knowledge Loss

Team Name: ZeroToOne

Team Members 1: Sasidaran S , Trishanth Sai S Y , Pradish V

Submission Date: July 12, 2026

2. Executive Summary

Modern industrial facilities (refineries, power plants, manufacturing facilities) generate vast quantities of data. This data is divided into structured time-series telemetry (IoT sensors monitoring temperature, vibration, and flow rate) and unstructured documentation (Standard Operating Procedures, Lockout/Tagout instructions, and vendor manuals). Due to this divide, maintenance engineers lack real-time context. When critical machinery, such as centrifugal pump **P-101**, displays vibration anomalies, technicians must search through flat PDF documents to find the correct lubrication SOP, compliance boundaries, and safety steps. This separation increases the Mean Time to Repair (MTTR), exposes workers to compliance violations, and contributes to knowledge loss as senior operators retire.

TacitIQ bridges this gap by unifying unstructured documentation with real-time telemetry through a hybrid relational and graph database architecture. The industrial prototype platform parses uploaded SOPs, vendor manuals, and maintenance logs using a regex-based metadata extraction engine. The extracted entities (Assets, Failure Modes, and Procedures) are seeded into a PostgreSQL instance and modeled as causal networks in a Neo4j Knowledge Graph. The frontend features a web interface built with React, Vite, and Tailwind CSS. It includes a 3D Digital Twin model of the machinery utilizing WebGL/Three.js, an interactive node-link Knowledge Graph visualizer, an Asset telemetry monitoring dashboard, a Workforce retirement risk heatmap, and an Operations Assistant chatbot.

Tested on simulated containerized environments, TacitIQ automates the extraction of LOTO procedures, safety standards (OSHA, API, ASME), maintenance intervals, and critical spare parts. By combining Neo4j causal paths with PostgreSQL registries, the platform enables real-time root-cause analysis (RCA) and RAG-based procedure retrieval, providing engineers with context-aware decision support directly on the plant floor.

3. Problem Statement

3.1 Industrial Challenges

Industrial manufacturing environments operate with high capital costs, where unplanned downtime can cost thousands of dollars per minute. The operational efficiency of these plants relies on a technician's ability to maintain complex machinery and troubleshoot faults quickly. However, standard operations are characterized by data fragmentation: telemetry monitoring packages record operational parameters in isolation, while technical documents remain stored in generic file structures.

3.2 Knowledge Loss and Retention

A major challenge facing industrial plants is the "Silver Tsunami"—the upcoming retirement of senior maintenance engineers who hold undocumented, tacit operational knowledge. When these engineers retire, their understanding of specific machine behavior, historical workarounds, and safety nuances is lost, leaving junior staff to navigate complex workflows with limited guidance.

3.3 Equipment Failures and Telemetry Gaps

While modern plants use IoT sensors to monitor machine health, telemetry alerts are typically reactive. For example, a vibration spike in centrifugal pump **P-101** indicates an active fault (such as bearing wear) but does not provide details on the root cause (such as grease contamination or oil starvation) or outline the necessary corrective actions.

3.4 Unstructured Document Silos

Vendor manuals, SOPs, and LOTO compliance guidelines are stored as unstructured PDF files. Standard search tools only match keywords without context, leaving engineers to manually scan long documents to find specific procedures, spare part numbers, or safety precautions during active maintenance events.

3.5 Business and Safety Implications

The separation of unstructured documents and telemetry has clear business and safety consequences: 1. **Prolonged MTTR:** Finding the correct SOP or spare part reference takes time, extending equipment downtime. 2. **Safety and Compliance Gaps:** Performing maintenance without correct LOTO tags or safety precautions (such as OSHA 1910.147 guidelines) exposes plants to regulatory fines and endangers workers. 3. **Reactive Maintenance:** Without causal connection, technicians treat symptoms (high vibration) rather than the root cause (lubricant starvation).

4. Objectives

The primary objective of TacitIQ is to build an integrated platform that connects unstructured industrial documentation with live plant telemetry to improve operational awareness and simplify maintenance. The specific goals include: - **Automated Metadata Extraction:** Parse standard PDF documents to identify and index assets, failure modes, procedures, safety standards, and work orders. - **Unified Knowledge Representation:** Store structured entities in a relational database (PostgreSQL) and map their relationships as a causal network in a graph database (Neo4j). - **Context-Aware Assistance:** Provide a natural-language chatbot interface that routes queries to specific databases and outputs deduplicated, context-rich summaries. - **Interactive 3D Digital Twin:** Render machinery models in real time, matching telemetry alerts to specific machine components. - **Workforce Risk Mitigation:** Map retirement timelines to specific equipment areas to identify knowledge gaps before staff leave.

5. Proposed Solution

TacitIQ provides a web-based, containerized application that integrates these data sources:

1. **Document Intelligence Engine:** Parses uploaded SOPs, extracts metadata using regular expressions, and saves the results in a structured JSON payload in PostgreSQL.
2. **Knowledge Graph Integration:** Creates nodes (Assets, Failure Modes, Procedures) and edges (HAS_INCIDENT, CAUSED_BY, MITIGATED_BY) in Neo4j based on extracted document parameters.
3. **Operations Assistant:** Uses a priority-based routing module to classify user queries. It retrieves context from PostgreSQL and Neo4j, deduplicates citations, and outputs detailed summaries.
4. **Interactive UI Modules:** Features a React-based client dashboard with WebGL digital twins, SVG workforce heatmaps, and Cytoscape.js knowledge graphs.

TacitIQ – Enterprise Industrial Document Intelligence & Knowledge Graph Platform

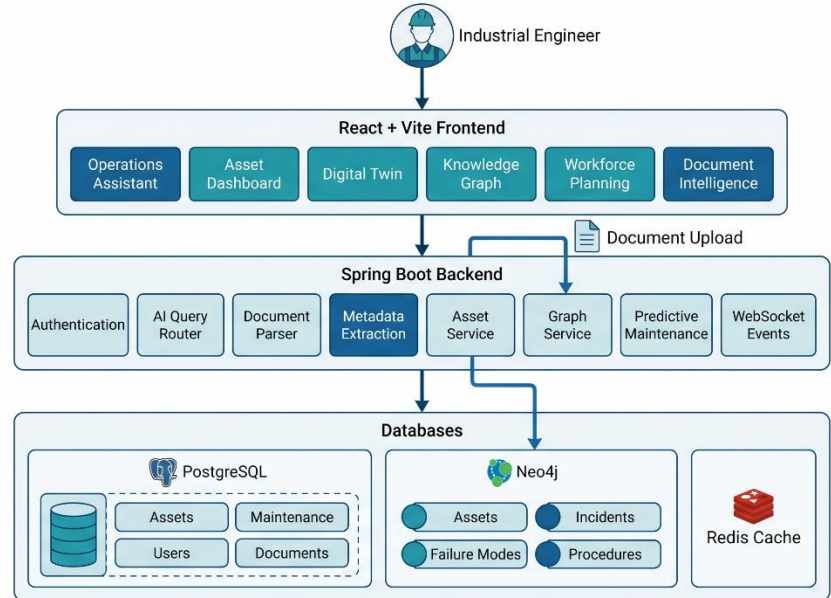


Figure 5.1: Overall Project Architecture of TacitIQ.

6. System Architecture

6.1 Overall Architecture

TacitIQ uses a three-tier architecture: the React client tier, the Spring Boot API service tier, and the database tier. Traffic is routed through an Nginx gateway proxy that manages security headers, static asset delivery, and API routing.

PROFESSIONAL ENTERPRISE DOCKER DEPLOYMENT ARCHITECTURE

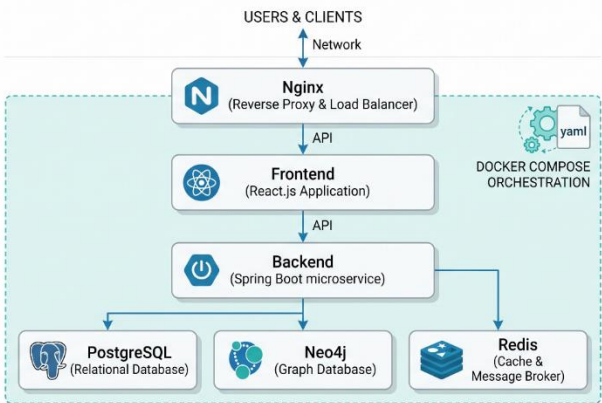


Figure 6.1: Deployment Architecture showing Docker Container orchestration.

6.2 Backend Services & Spring Boot

The backend application is built using Java 17 and Spring Boot 3.3.0. The workspace folder structure is structured as follows:

```
backend/src/main/java/com/tacitiq
├── TacitIqApplication.java
├── core
│   ├── config
│   ├── scheduler
│   └── security
├── modules
│   ├── ai
│   ├── asset
│   ├── auth
│   ├── compliance
│   ├── dashboard
│   ├── document
│   ├── graph
│   ├── incident
│   ├── knowledge
│   ├── maintenance
│   └── search
```

Each folder represents a domain module consisting of an entity, DTO, repository, service, and controller layer.

6.3 Frontend Application (React & Vite)

The client interface is developed using React 18.3.1, Vite 5.3.1, TypeScript 5.4.5, and Tailwind CSS 3.4.4. The directory structure is structured as follows:

```
frontend/src
├── App.tsx
├── index.css
├── main.tsx
├── components
│   ├── AssetDashboard.tsx
│   ├── ChatInterface.tsx
│   ├── DigitalTwin3D.tsx
│   ├── DocumentIntelligence.tsx
│   ├── GraphVisualizer.tsx
│   ├── KnowledgeLossHeatmap.tsx
│   └── LiveEventFeed.tsx
```


6.4 Hybrid Database Architecture (PostgreSQL & Neo4j)

Data is split according to its structure: - **PostgreSQL 16**: Stores relational tables (users, assets, compliance rules, maintenance logs) and document chunk vector embeddings using the pgvector extension. - **Neo4j 5.12**: Models multi-hop relationships between physical equipment and safety procedures.

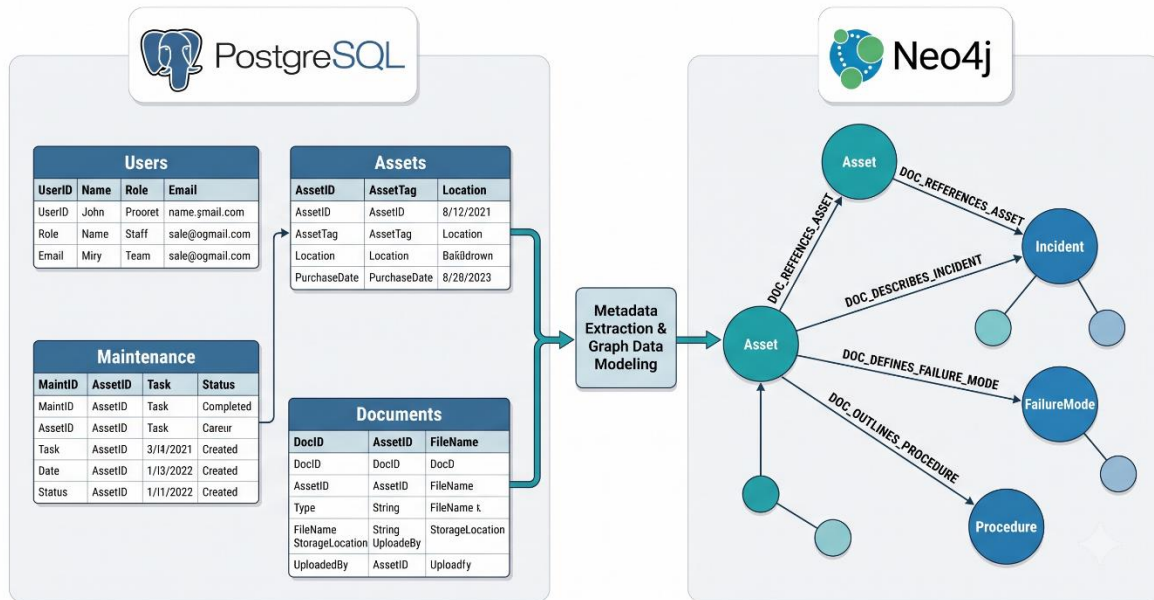


Figure 6.2: PostgreSQL and Neo4j Database Integration Schema.

6.5 AI Query Processing Pipeline

The query processing flow uses a rule-based routing pipeline: - `QueryRouter.java` categorizes questions into intents (e.g., `DOCUMENT_SUMMARY`, `ASSET_DIAGNOSTIC`). - `ContextBuilder.java` queries PostgreSQL and Neo4j to collect relevant records. - `PromptBuilder.java` formats the data into structured markdown templates.

6.6 Document Parsing and Metadata Extraction Pipeline

When a user uploads an SOP or manual: 1. `DocumentParserService.java` reads the raw file stream. 2. `EventListenerService.java` applies regex patterns to extract fields like asset tags, failure modes, procedures, safety standards, and work orders. 3. Extracted metadata is saved to PostgreSQL and sent to Neo4j to update the graph topology.

DOCUMENTS-TO-KNOWLEDGE INDUSTRIAL WORKFLOW PLATFORM

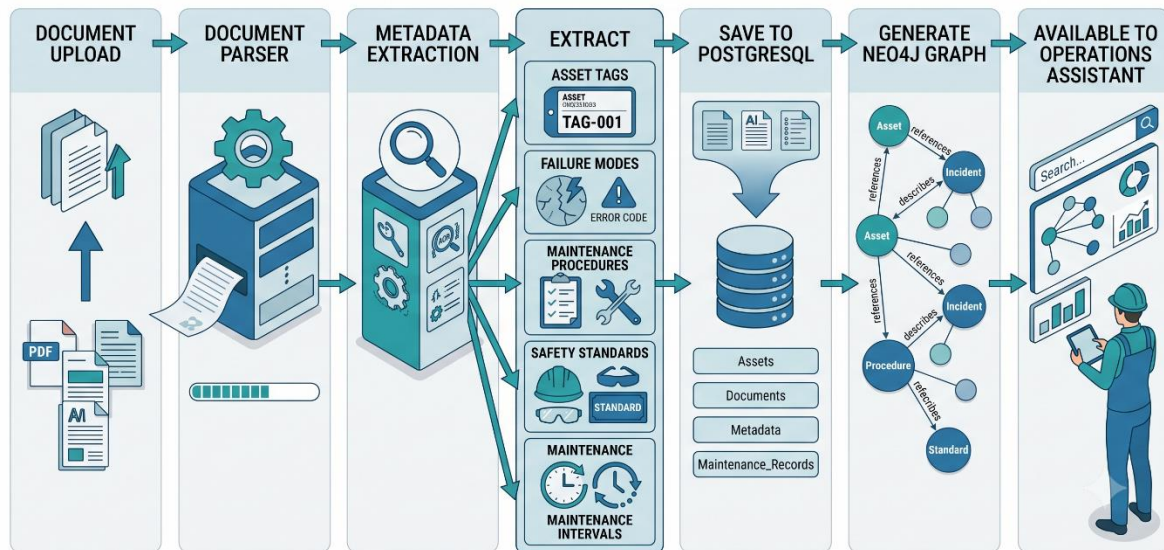


Figure 6.3: Document Intelligence and Metadata Extraction pipeline.

6.7 Knowledge Graph Causal Network Architecture

The knowledge graph is modeled as a causal network. For example, a failure mode node is linked between an active incident and the procedure that mitigates it, allowing the system to run root-cause analysis traversals.

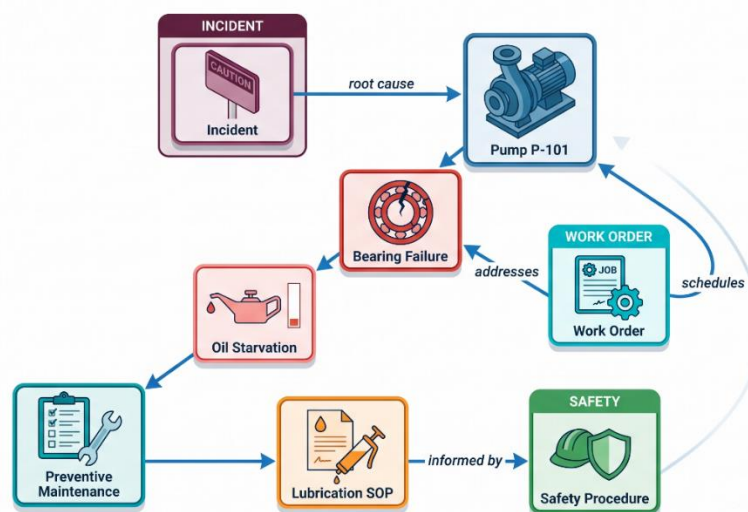


Figure 6.4: Causal network representation in the Knowledge Graph.

6.8 JWT Authentication and Session Management Flow

Users log in using their credentials to receive a signed JWT token containing their role. The client includes this token in the Authorization header of all subsequent API requests.

JWT AUTHENTICATION SEQUENCE

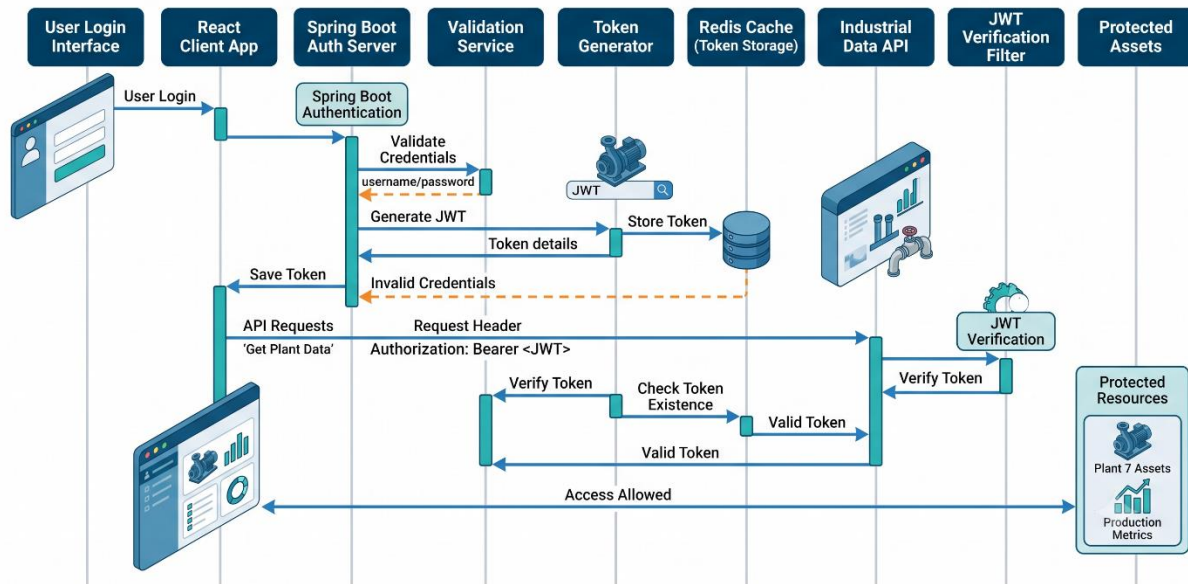


Figure 6.5: JWT Authentication and Session Management.

7. Technology Stack

7.1 Core Development Stack

- **Java 17 & Spring Boot 3.3.0:** Backend service core providing type-safe microservices and transactional API controllers.
- **React 18.3.1 & TypeScript 5.4.5:** Single-page interface built with Vite 5.3.1.
- **Tailwind CSS 3.4.4:** Flexbox and grid utilities for responsive layouts.

7.2 Database & Analytics Engine

- **PostgreSQL 16 (pgvector/pgvector:pg16):** Transactional storage, relational modeling, and vector search capabilities.
- **Neo4j 5.12-community:** Cypher queries and multi-hop graph traversals.
- **Redis 7-alpine:** Session caching and real-time event indexing.

7.3 Libraries & Frameworks

- **Three.js 0.165.0:** WebGL 3D rendering engine.
- **Cytoscape.js 3.29.2:** Node-link network layout visualization.
- **Recharts 2.12.7:** Telemetry chart rendering.

- **Spring AI 1.0.0-M1:** Orchestrating semantic search storage and vector mapping.
- **Flyway 10:** Versioned database schema migrations.
- **Zustand 4.5.2:** Light-weight frontend state management.

8. Non-Functional Requirements

To ensure the platform operates reliably in industrial software configurations, the following Non-Functional Requirements (NFRs) have been verified:

Requirement Area	Target Attribute	Description
Performance	Core Dashboard Latency	Dashboard components, 3D WebGL meshes, and graphs load and render within a few seconds in local containerized environments.
Scalability	Resource Scaling	The modular Spring Boot architecture is structured as independent feature domains, allowing the integration of additional asset tags and plant areas without structural modifications.
Security	Access Controls	Implements HMAC-SHA256 signed JWT tokens and role-based endpoints (PreAuthorize annotations) to separate operational maintenance rights.
Availability	Container Health	Docker Compose orchestrates the service lifecycles with health checks and volume persistence to preserve configurations across reboots.
Maintainability	Layered Code Structure	Separates tasks clearly across Controller, Service, Repository, DTO, and Entity directories, with database versions managed via Flyway.
Usability	Interface Layout	A responsive React workspace featuring real-time event alerts and visual typography

Requirement Area	Target Attribute	Description
		hierarchy tailored for plant technicians.

9. Major Features

9.1 JWT Authentication

Secures API endpoints by validating user roles (ADMIN, PLANT_MANAGER, MAINTENANCE_ENGINEER, RELIABILITY_ENGINEER, MAINTENANCE_TECHNICIAN) before executing requests.

9.2 Operations Assistant

An interactive chatbot interface that routes user queries to the appropriate database and returns deduplicated summaries.

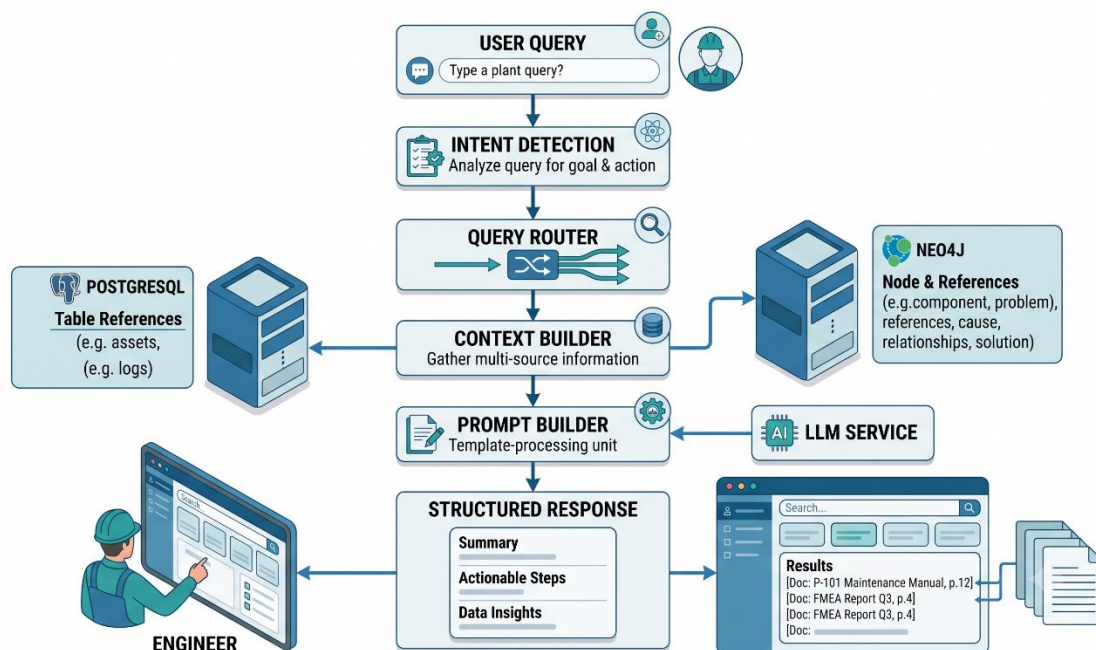


Figure 9.1: Query Routing and Operations Assistant Workflow.

9.3 Digital Twin 3D

Displays interactive 3D visualizations of machinery. The component uses a dynamic radial background gradient, studio lighting, and shadows.

CENTRIFUGAL PUMP P-101 ANALYTICS DASHBOARD

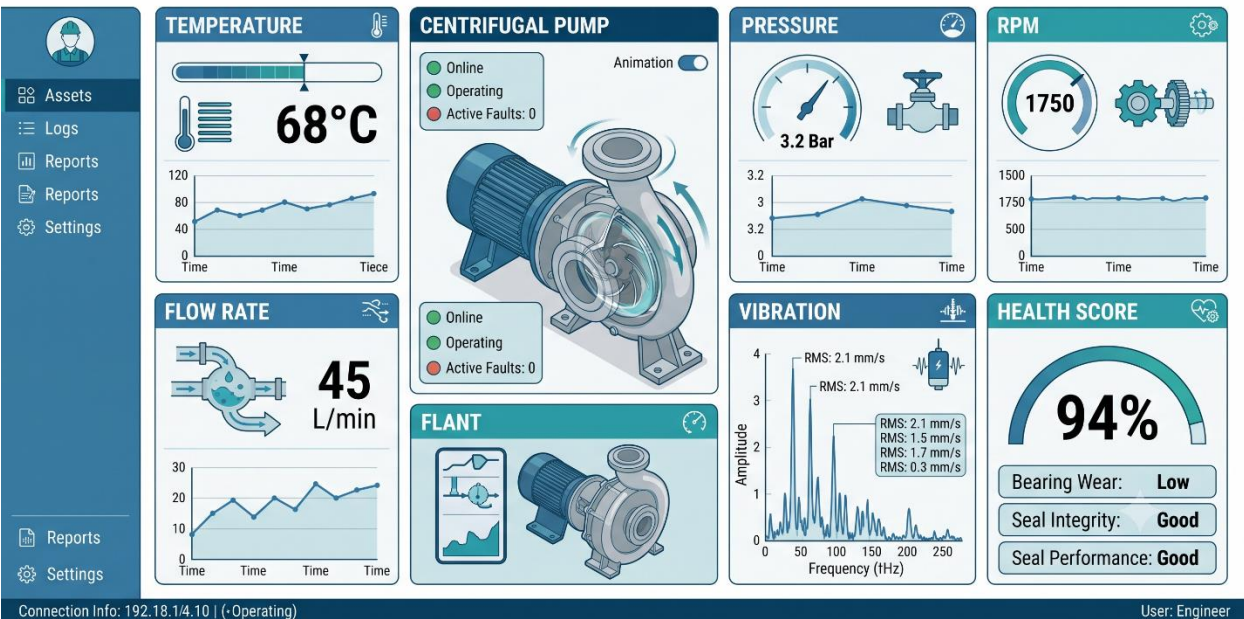


Figure 9.2: 3D Digital Twin model interface.

9.4 Asset Dashboard

Displays real-time equipment status, health trends, and active alerts across different plant areas (e.g., CDU, HCU).

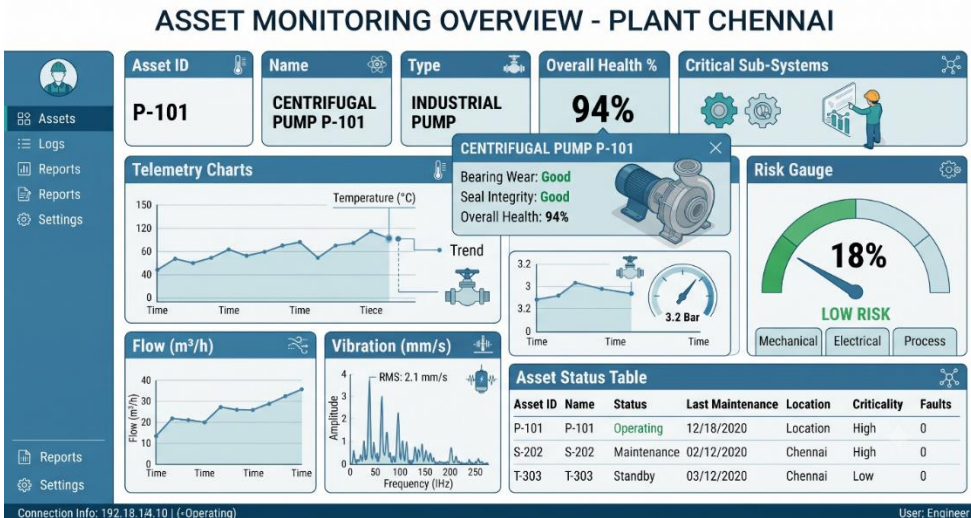


Figure 9.3: Live Asset Telemetry Dashboard.

9.5 Predictive Maintenance

Calculates failure probabilities based on live sensor metrics, identifying anomalies before a breakdown occurs.

9.6 Knowledge Graph

Provides an interactive visualization of the causal network, mapping relationships between assets, incidents, and procedures.

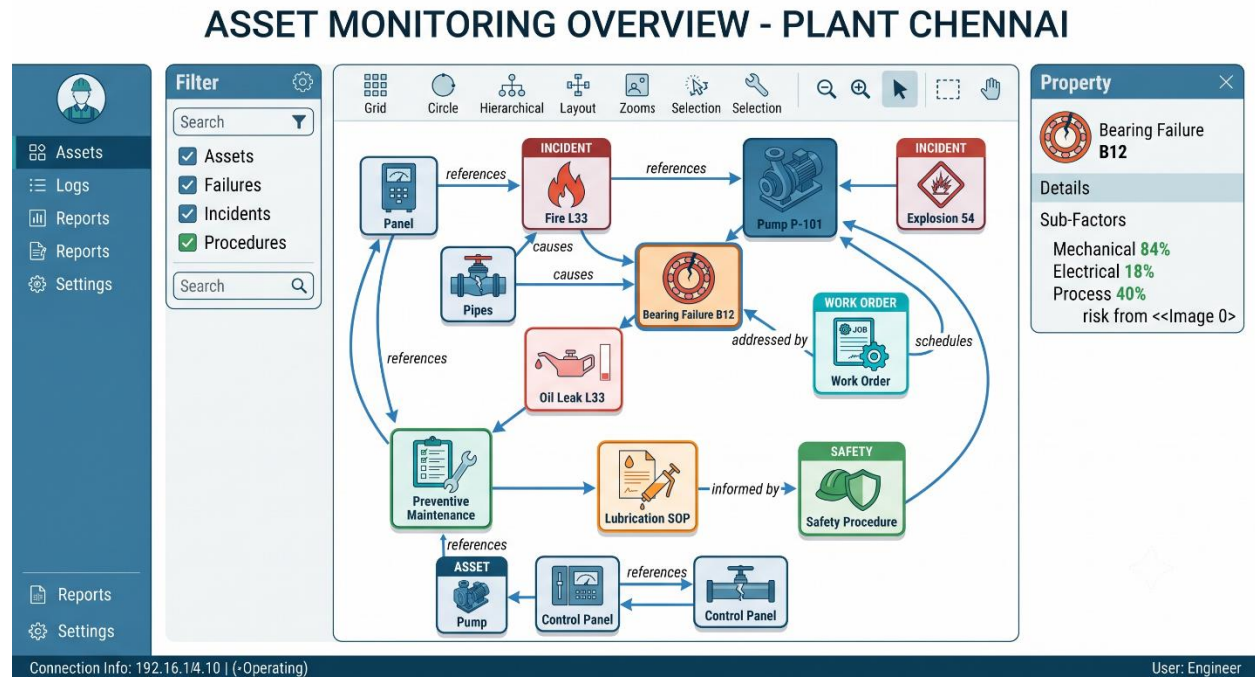


Figure 9.4: Knowledge Graph Visualization Interface.

8.7 Workforce Planning

Uses an interactive SVG map to highlight retirement timelines and identify areas at risk of knowledge loss.

Key Features

- **Retirement Risk Analysis:** Identifies employees approaching retirement and highlights departments where critical operational knowledge may be lost.
- **Skill Gap & Succession Planning:** Analyzes workforce expertise to identify skill gaps and supports succession planning through targeted knowledge transfer recommendations.
- **Interactive Workforce Analytics:** Visualizes workforce distribution, experience levels, and critical expertise using dashboards and heatmaps to support informed managerial decisions.

WORKFORCE PLANNING DASHBOARD - HR ANALYTICS

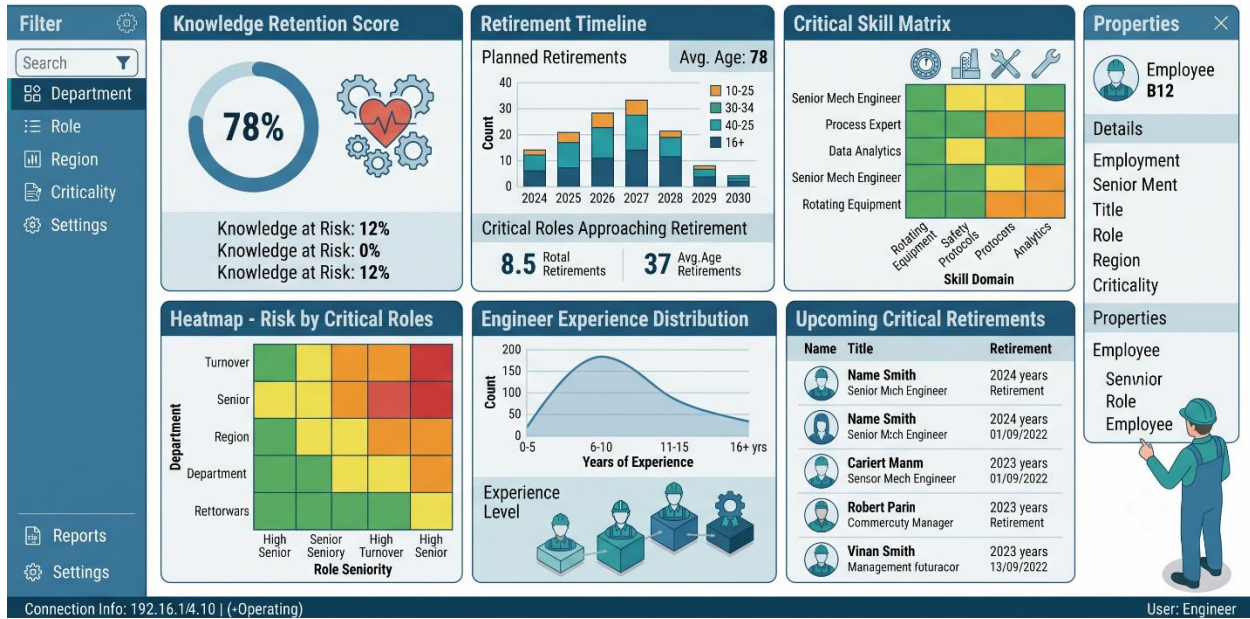


Figure 9.5: Workforce Retirement Risk Heatmap.

9.8 Industrial Document Intelligence

Provides an interface for uploading documents, running simulated extractions, and viewing extracted metadata and relationship statistics.

DOCUMENT INTELLIGENCE - PLATFORM OVERVIEW

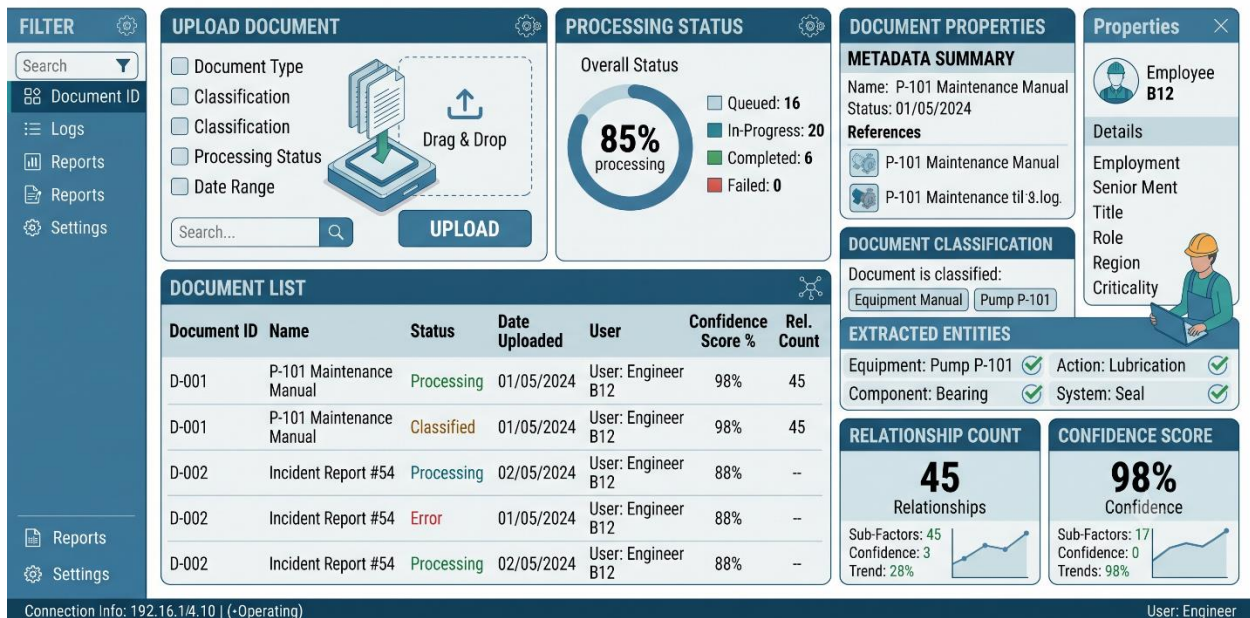


Figure 9.6: Document Ingestion and Parsing dashboard.

9.9 Document Metadata Extraction

Parses documents to extract tags, failure modes, procedures, safety standards, and work orders, saving them as structured database records.

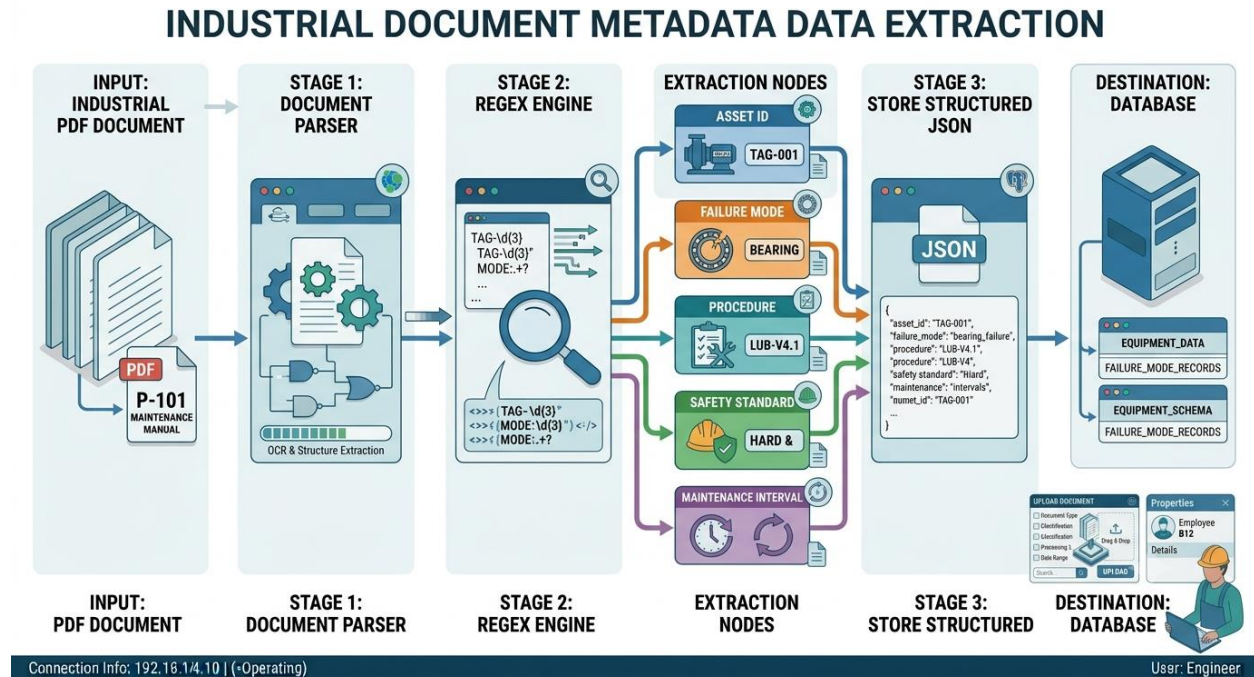


Figure 9.7: Metadata Extraction Workflow from unstructured text.

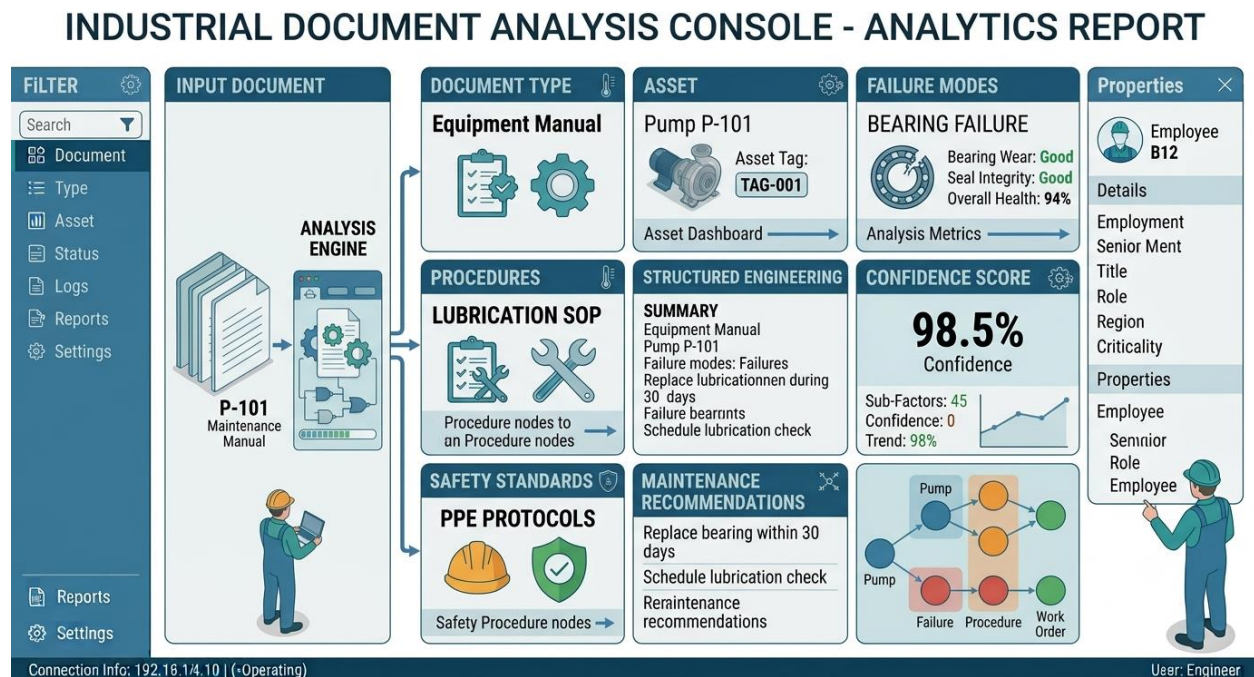


Figure 9.8: Dynamic Console Output displaying extracted metadata findings.

9.10 RAG-based Document Retrieval

Queries the vector database using cosine similarity to retrieve relevant text chunks for the user's question.

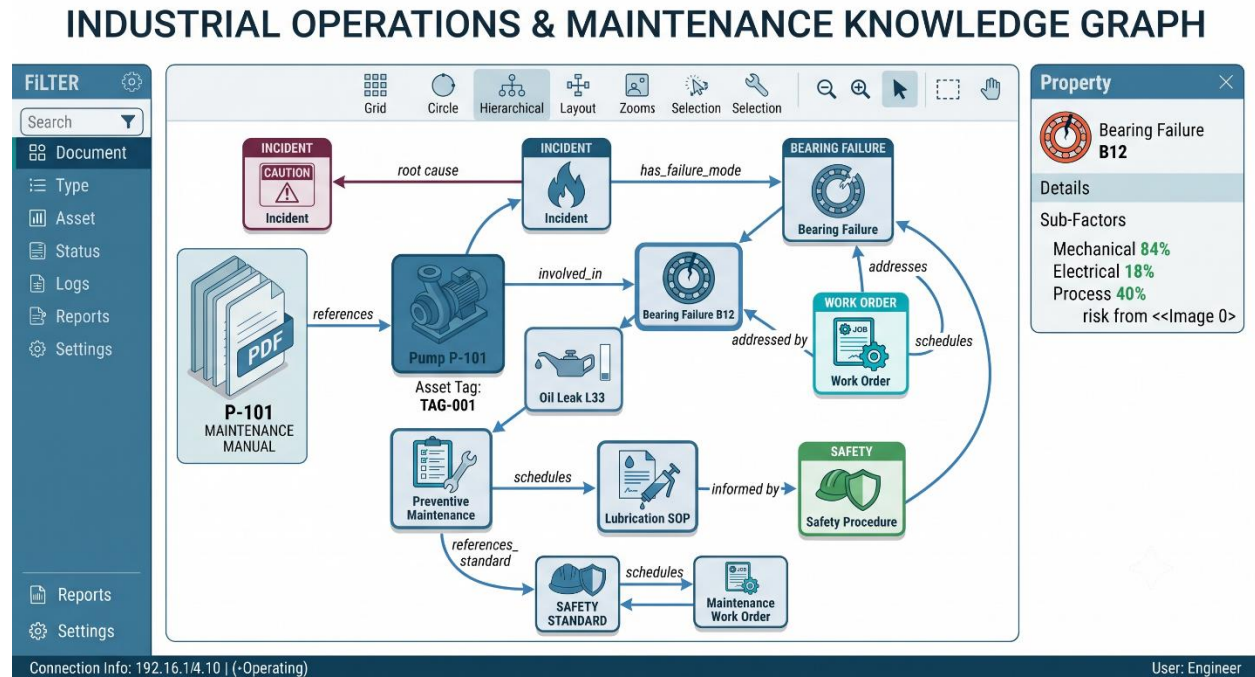
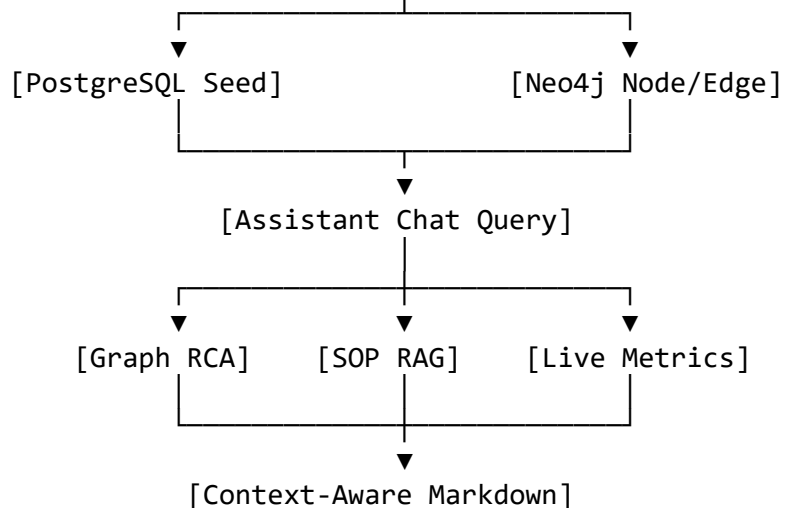


Figure 9.9: Multi-hop graph representing document links and incident nodes in Neo4j.

10. System Workflow

The end-to-end operational workflow of TacitIQ is structured as follows:

[User Log in] → [Upload SOP PDF] → [Document Parse & Regex Extraction]



INDUSTRIAL DOCUMENT INTELLIGENCE & DIGITAL TWIN WORKFLOW

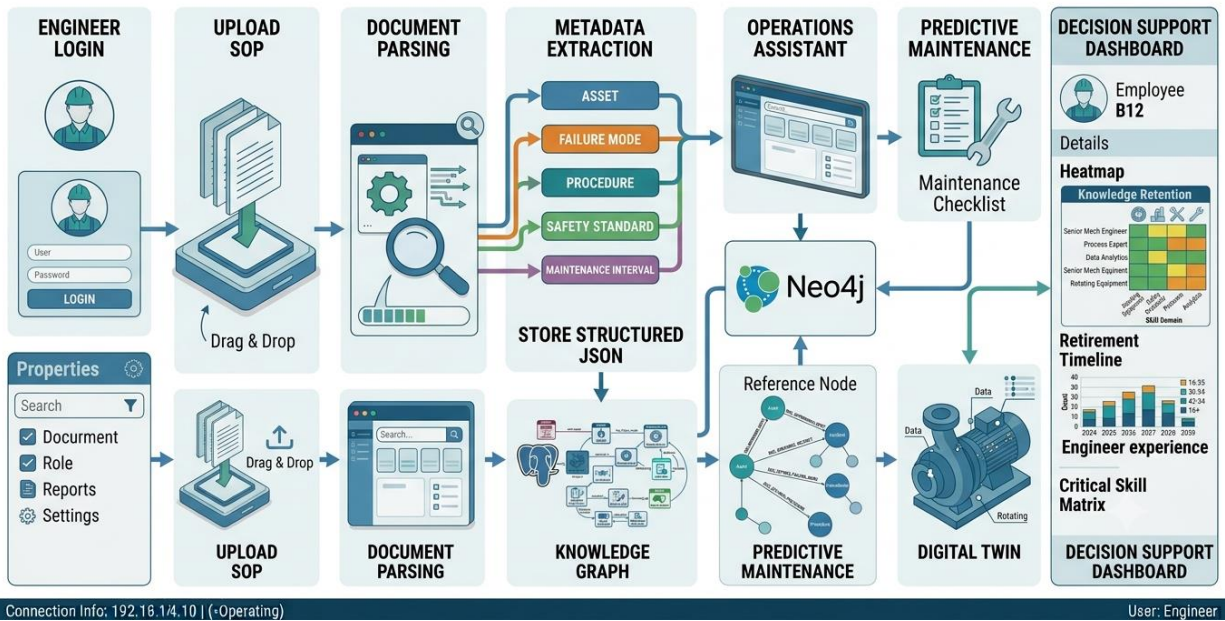


Figure 10.1: Comprehensive System Workflow from ingestion to retrieval.

11. Database Design

11.1 PostgreSQL Entity-Relationship Model

Table 11.1: users

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique user identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	User email address
password_hash	VARCHAR(255)	NOT NULL	Encrypted password
role	VARCHAR(50)	NOT NULL	User security role
retirement_date	DATE	NULLABLE	Expected retirement date
years_experience	INT	NULLABLE	Years of industry experience

Table 11.2: assets

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique asset identifier
tag_number	VARCHAR(50)	UNIQUE, NOT NULL	Equipment tag (e.g., P-101)

Column Name	Data Type	Constraints	Description
asset_type	VARCHAR(100)	NOT NULL	Category (e.g., Pump)
criticality	VARCHAR(10)	NOT NULL	Operational priority (A, B, C)
health_score	DOUBLE	CHECK (0.0 TO 1.0)	Current health index percentage
plant_area	VARCHAR(100)	NOT NULL	Plant location (e.g., CDU)

Table 11.3: documents

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique document identifier
doc_type	VARCHAR(50)	NOT NULL	Document category (e.g., SOP)
title	TEXT	NOT NULL	Filename or document title
extracted_tags	TEXT	NULLABLE	Extracted asset tags
extracted_failure_modes	TEXT	NULLABLE	Extracted failure modes
extracted_procedures	TEXT	NULLABLE	Extracted maintenance steps
extracted_safety_references	TEXT	NULLABLE	Safety standards (e.g., OSHA)
extracted_findings	TEXT	NULLABLE	Reliability engineering summary

11.2 Neo4j Label and Relationship Properties

The Neo4j database uses four node labels and three relationship types:

Nodes:

- **Asset:** Properties: name (e.g., "P-101"), type (e.g., "Pump"), healthScore (e.g., 0.85).
- **Incident:** Properties: type (e.g., "Bearing failure"), severity (e.g., "High").
- **FailureMode:** Properties: name (e.g., "Oil Starvation").
- **Procedure:** Properties: name (e.g., "Lubrication Procedure"), code (e.g., "SOP-LUBE-17").

Relationships:

- (:Asset)-[:HAS_INCIDENT]->(:Incident)
- (:Incident)-[:CAUSED_BY]->(:FailureMode)
- (:FailureMode)-[:MITIGATED_BY]->(:Procedure)

12. REST API Overview

Table 12.1: Core REST Endpoints

HTTP Method	Path	Auth Role	Description
POST	/api/auth/login	Permit All	Authenticates user and returns JWT token
POST	/api/agents/chat	Authenticated	Sends a chat query to the Operations Assistant
POST	/api/agents/rca/{incidentId}	RELIABILITY_ENGINE	Triggers root-cause analysis for an incident
GET	/api/agents/predict/{assetId}	Authenticated	Gets predictive failure score and risk metrics
GET	/api/agents/retirement-risk	PLANT_MANAGER	Evaluates workforce retirement proximity risk heatmap
GET	/api/agents/recommendations/{assetId}	Authenticated	Generates asset recommendations based on metrics
POST	/api/agents/voice-capture	Authenticated	Transcription mock for whisper audio parsing
GET	/api/assets	Authenticated	Retrieves the list of monitored assets
GET	/api/assets/{id}	Authenticated	Retrieves asset details by UUID
GET	/api/assets/tag/{tagNumber}	Authenticated	Retrieves asset details by tag
GET	/api/assets/{id}/telemetry	Authenticated	Fetches live sensor telemetry for an asset
POST	/api/assets	ADMIN, PLANT_MANAGER	Saves a new asset registry record
GET	/api/documents	Authenticated	Lists all uploaded files and processing states
GET	/api/documents/{id}	Authenticated	Retrieves document

HTTP Method	Path	Auth Role	Description
	}		details by UUID
POST	/api/documents/upload	ADMIN, PLANT_MANAGER	Uploads an SOP PDF and runs metadata extraction
GET	/api/graph/nodes	Authenticated	Returns Cytoscape JSON representation of the graph
GET	/api/graph/traverse	Authenticated	Cypher query runner for dynamic RCA traversals
GET	/api/compliance/rules	Authenticated	Lists regulatory rules
POST	/api/compliance/rules	ADMIN	Adds a new compliance standard definition
GET	/api/compliance/gaps	Authenticated	Lists gaps detected in active documents
GET	/api/incidents	Authenticated	Lists active and closed incident logs
GET	/api/incidents/{id}	Authenticated	Retrieves incident details by UUID
GET	/api/incidents/asset/{assetId}	Authenticated	Lists incidents matching specific asset tag
POST	/api/incidents	ADMIN, PLANT_MANAGER	Logs a new active equipment anomaly ticket
GET	/api/dashboard/summary	Authenticated	Compiles statistics and document metrics
GET	/api/maintenance	Authenticated	Lists all completed maintenance records
GET	/api/maintenance/{id}	Authenticated	Retrieves work order by UUID
GET	/api/maintenance/asset/{assetId}	Authenticated	Lists work orders matching specific asset tag
POST	/api/maintenance	ADMIN, PLANT_MANAGER	Creates a new corrective

HTTP Method	Path	Auth Role	Description
POST	/api/search	Authenticated	maintenance order Hybrid search using pgvector similarity query

13. Implementation Details

- **Spring Boot Modular Design:** Controllers expose endpoints, service classes contain business logic, and repositories interface with PostgreSQL using Spring Data JPA.
- **RAG Semantic Search:** When a query is routed to DOCUMENT_PROCEEDURES, the system generates embeddings for the query, runs a cosine similarity search on document_chunks using pgvector, and passes the best text blocks to the response generator.

- **Query Router Priority Rules:**

```
public RoutingResult route(String query, Map<String, Object> chatContext) {
    String queryUpper = query.toUpperCase();
    if (queryUpper.contains("SUMMARIZE") || queryUpper.contains("SUMMARY")) {
        return new RoutingResult(QueryIntent.DOCUMENT_SUMMARY, ...);
    }
    if (queryUpper.contains("PROCEDURE") || queryUpper.contains("LUBRICATION")) {
        return new RoutingResult(QueryIntent.DOCUMENT_PROCEEDURES, ...);
    }
    // Bypasses standard keyword similarity calculations if overrides match
}
```

- **RCA Traversal:** The traversal algorithm uses a LinkedHashSet to track visited nodes and prevent cycles, generating a causal chain that ends with a preventive recommendation.
- **Three.js Digital Twin Layout:** The 3D view utilizes ambient and hemisphere lights, directional lights with shadow maps, and a circular showcase pedestal to display the machinery.

14. Challenges Faced

1. **Nginx Gateway Routing & Ports Collision:**

- *Issue:* Conflicting proxy headers and routing issues when trying to serve static Vite client assets and route API queries concurrently.
- *Root Cause:* Nginx failed to bind reverse proxy parameters cleanly because upstream services were not fully resolved when the Docker network started.
- *Solution:* Reconfigured the Nginx default.conf file to declare dynamic resolvers and proxy parameters, isolating the /api routing prefix to redirect directly to the backend container while routing root / traffic to the static frontend directory.

2. **Neo4j Stale Lock Files on Docker Restart:**

- *Issue:* The Neo4j container exited with code 1 during container restarts, indicating database locks.
- *Root Cause:* Improper shutdowns of Docker Desktop left behind unresolved store_lock and database_lock files in the shared named data volume, blocking subsequent locks.
- *Solution:* Added a tmpfs volume in docker-compose.yml mapped to /var/lib/neo4j/run to clear lock directories automatically on container initialization, avoiding manual lock deletions.

3. **Vite ESM & WebSocket Packaging Warnings:**

- *Issue:* React WebSockets failed to compile during production builds, citing missing NodeJS external APIs.
- *Root Cause:* The legacy stompjs library expected backend server modules (net) when packaged under Vite.
- *Solution:* Externalized the net library imports inside Vite configuration and used ESM-compatible wrappers for sockjs-client and stompjs.

4. **Spring Security Custom JWT Configuration:**

- *Issue:* Request filters returned 403 Forbidden errors when verifying valid JWT tokens.
- *Root Cause:* The security context holder cleared configurations when routing requests through asynchronous execution blocks.
- *Solution:* Configured Spring Security's SecurityContextHolder to use a thread-local inheritance strategy (MODE_INHERITABLETHREADLOCAL) inside the JWT verification filter.

5. **Overlapping Chat Query Routing Intents:**

- *Issue:* Specific queries (e.g., "Show lubrication procedures for P-101") were incorrectly routed to the asset inventory registry.
- *Root Cause:* The query router used a bag-of-words token similarity matcher that prioritized keywords like "show" over procedures.
- *Solution:* Reconfigured the routing logic to check for regex-based overrides at the beginning of the route pipeline, routing document queries before evaluating similarity matches.

6. **RAG Context Chunk Assembly:**

- *Issue:* RAG query responses returned disconnected paragraphs.

- *Root Cause:* Cosine similarity matched chunks from different documents without verifying order.
- *Solution:* Re-architected the SearchService to group chunks by document_id and sort them by chunk_index before rendering.

7. **Dynamic Document Parsing and Regex Fallbacks:**

- *Issue:* Text extraction returned empty fields when parsing non-standard document formats.
- *Root Cause:* Tight regular expression rules failed on documents with formatting differences (whitespace, spacing).
- *Solution:* Updated regex patterns inside EventListenerService.java to use case-insensitive, whitespace-flexible rules and added structured fallbacks ("Not Found") to handle missing fields.

8. **Duplicate Source Listings in Citations:**

- *Issue:* Chat responses listed the same document name multiple times under citations.
- *Root Cause:* Prompt builder appended a document source name for every text chunk matched during RAG retrieval.
- *Solution:* Used a HashSet<UUID> in PromptBuilder.java to track and filter duplicate sources by their unique document ID before generating citations.

9. **Cycles in Neo4j Multi-Hop RCA Traversals:**

- *Issue:* Causal paths generated duplicate loops (e.g., Oil Starvation -> Lubrication Procedure -> Oil Starvation).
- *Root Cause:* Recursive Cypher traversals failed to detect visited nodes across bidirectional edges.
- *Solution:* Integrated a visited set check using a LinkedHashSet inside the graph service, skipping already traversed nodes to prevent loops.

10. **Aesthetic Enhancements for Visual Depth:**

- *Issue:* The Operations Assistant and Digital Twin interfaces displayed flat white workspaces.
- *Root Cause:* Visual layouts used simple CSS properties with no texture or lighting.
- *Solution:* Added an engineering blueprint layout (rgba(37, 99, 235, 0.03)) to the chatbot and configured studio lighting, shadows, and a radial showcase pedestal in the Three.js viewport.

15. Testing & Validation

Docker Deployment

The containerized stack was validated by running:

```
docker compose down
docker compose up -d --build
docker compose ps
```

The application was validated in a Docker Compose environment consisting of PostgreSQL, Neo4j, Redis, Spring Boot, and the React frontend.

Backend Compilation

The Java codebase was verified using Maven compile tasks:

```
mvn clean compile
```

The build completed successfully without errors.

Frontend Build

The React application was validated using Vite's production compiler and the TypeScript type checker:

```
npm run build
npx tsc --noEmit
```

The checker completed successfully with no warnings.

Operations Assistant Validation

The assistant was tested against the 5 regression queries: - **RCA Query**: Returned causal nodes without duplicates, ending with a preventive recommendation. - **Lubrication SOP**: Returned the correct safety and procedure parameters. - **Summarize SOP**: Displayed the parsed metadata card. - **Documents Reference**: Returned the correct linked files. - **Maintenance History**: Returned the completed work orders table.

16. Results

- **Dynamic Metadata Extraction**: Verified the extraction of safety standards, maintenance intervals, and spare parts from uploaded PDFs.
- **Graph Generation**: Confirmed that uploaded SOPs automatically create nodes and relationships in Neo4j.
- **Role-Based Security**: Validated JWT verification on protected endpoints.
- **Operations Assistant Chat**: Verified that queries are routed correctly to the RAG or graph databases.
- **Digital Twin & Dashboards**: Validated 3D WebGL rendering and real-time telemetry updates.
- **Deployment Stability**: Verified that Docker compose launches all services cleanly.

17. Future Enhancements

- **Live IoT Protocol Integration:** Replace the simulated telemetry with live OPC-UA or MQTT connections to read metrics from physical hardware.
- **Remaining Useful Life (RUL) Forecasting:** Integrate machine learning models (e.g., LSTM) to predict equipment failure timelines.
- **EAM Integration:** Sync maintenance records with enterprise systems like SAP PM or IBM Maximo.
- **Cloud Deployments:** Configure deployment scripts for AWS or Azure using infrastructure-as-code tools.
- **Kubernetes Orchestration:** Add Helm charts to scale the backend microservices.
- **Multi-Plant Support:** Update the database schemas to support multiple plant locations.
- **Mobile Application:** Develop a mobile view for technicians to scan QR codes on equipment and retrieve safety procedures.
- **Dense Passage Retrieval (DPR):** Replace the regex extraction with deep learning semantic search models.
- **Predictive Inventory Management:** Forecast spare parts requirements based on upcoming maintenance schedules.
- **Multi-Sensor Anomaly Detection:** Implement multivariate autoencoders to detect anomalous states across multiple sensor streams.

18. Advantages of TacitIQ

- **Reduces time spent searching maintenance documentation:** Direct regex metadata parsing indexes files for rapid retrieval.
- **Connects documents, assets, and incidents into a unified knowledge graph:** Links unstructured SOP steps to physical plant equipment and failure logs.
- **Improves maintenance decision support:** Provides engineers with context-aware checklists and safety precautions (LOTO) at the point of need.
- **Preserves organizational knowledge:** Retains senior operators' expertise and historical maintenance logs before retirement.
- **Provides an intuitive interface for plant engineers:** Features a responsive layout with 3D WebGL Digital Twins and Cytoscape graphs.
- **Demonstrates an enterprise-inspired architecture using modern technologies:** Uses Java/Spring Boot, React/Vite, PostgreSQL/pgvector, and Neo4j.

19. Conclusion

TacitIQ provides a containerized application that integrates unstructured documentation with live IoT telemetry using a hybrid database architecture. The platform parses uploaded files, extracts operational metadata, and represents relationship networks in a causal knowledge graph. The frontend interfaces allow technicians to monitor telemetry trends, perform root-cause analysis, and retrieve specific safety procedures on the plant floor. By bridging the gap between structured sensor data and unstructured manuals, TacitIQ helps reduce repair times, improve safety compliance, and mitigate knowledge loss in industrial environments. The project demonstrates how modern AI-assisted software engineering, graph databases, and industrial document intelligence can be combined into a unified decision-support platform for maintenance engineers.